УчИМИ Ученически институт по математика и информатика

ДВАДЕСЕТ И ШЕСТА УЧЕНИЧЕСКА СЕСИЯ
УС'26

ТЕМА НА ПРОЕКТА

HASHMOUTH – ПРОТОКОЛ ЗА АНОНИМНА КОМУНИКАЦИЯ

Автори:

Самуил Станиславов Маринов, ПМГ „Иван Вазов“, Добрич, 12 клас
Славина Илкова Цонева, ПМГ „Иван Вазов“, Добрич, 12 клас

Научен ръководител:

Светлана Василева, учител, ПМГ „Иван Вазов“, Добрич

СЪДЪРЖАНИЕ

РЕЗЮМЕ 3
ВЪВЕДЕНИЕ 4
КРАТЪК ОБЗОР НА СЪЩЕСТВУВАЩИТЕ СИСТЕМИ – ПРЕДИМСТВА И НЕДОСТАТЪЦИ 5
АРХИТЕКТУРА И МЕТОДОЛОГИЯ 7
ТЕХНИЧЕСКА РЕАЛИЗАЦИЯ 8
ЗАКЛЮЧЕНИЕ И БЪДЕЩА РАБОТА 9
ИЗТОЧНИЦИ 9

РЕЗЮМЕ

HashMouth е децентрализирана P2P комуникационна система, реализираща многослойно (onion) криптиране, многопътен (multi-path) routing и механизми за намаляване на метаданните. Целта е да се подобри анонимността и производителността спрямо съществуващите решения (*Tor*, I2P, mix-мрежи), като се използват техники като многопътен маршрут, разпределени *P2P* канали (включително адаптация на механики от *BitTorrent*) и силна криптография с периодична смяна на сесийните ключове (ratchet). Документът съдържа критичен преглед на слабостите на конкурентните системи, предложение за архитектура на HashMouth и конкретни защитни мерки, подкрепени с публикувани изследвания.

SUMMARY

HashMouth is a decentralized P2P communication system implementing multi-layer (onion) encryption, multi-path routing, and metadata reduction mechanisms. The goal is to improve anonymity and performance compared to existing solutions (Tor, I2P, mix networks) by using techniques such as multi-path routing, distributed P2P channels (including adaptations of BitTorrent mechanics), and strong cryptography with periodic session key rotation (ratchet). The document contains a critical review of the weaknesses of competing systems, a proposed architecture for HashMouth, and specific protective measures supported by published research.

ВЪВЕДЕНИЕ

В съвременния дигитален свят проблемите, свързани със сигурност, анонимност и децентрализация на комуникациите, придобиват все по-голямо значение. Проекти като *Tor*, I2P и Freenet се стремят да осигурят поверителност и защита на данните, но страдат от редица проблеми – бавна скорост, централизация на възлите, лесна идентификация на трафика и уязвимости при корелационни атаки. *HashMouth* е нова експериментална децентрализирана комуникационна система, която има за **цел**: да комбинира сигурността на *Tor* с бързината на *P2P* мрежите, използвани при торенти; да осигури реално криптирана и маршрутизирана комуникация между възли; да позволи пренос на реални протоколи като *HTTP*, *HTTPS*, *WebRTC*; да поддържа уеб приложения за комуникация, подобни на Messenger или Skype, върху анонимна инфраструктура. За постигане целта на проекта е необходимо да се решат следните задачи:

1. Да се създаде децентрализирана мрежа от равнопоставени възли (peers)**,** които могат да комуникират директно помежду си.
2. Да се внедри многостепенно (onion) криптиране, което защитава данните по пътя им през мрежата.
3. Да се реализира надежден протокол за маршрутизация, при който всеки възел познава само предходния и следващия участник.
4. Да се постигне високо ниво на сигурност и анонимност чрез комбинация от асиметрично и симетрично криптиране.
5. Да се демонстрира практическа имплементация на криптографски алгоритми в Go.

КРАТЪК ОБЗОР НА СЪЩЕСТВУВАЩИТЕ СИСТЕМИ – ПРЕДИМСТВА И НЕДОСТАТЪЦИ

1. *Tor*

   Силни страни: широко разпространение, удобен клиент, поддръжка на onion услуги. Слаби страни: уязвимост към анализ на трафика (особено глобален пасивен наблюдател), възможни деанонимизиращи атаки и проблеми при използване на *P2P* приложения върху *Tor* (*BitTorrent* може да издаде истинския IP). Това е документирано в множество публикувани статии и изследвания за трафиков анализ и деанонимизиране (css.csail.mit.edu+1).

2. *I2P*

   Силни страни: децентрализиран подход с *DHT* (*netDB*) и адаптиран за вътрешни (*hidden*) услуги.
   Слаби страни: *DHT* базираната архитектура понякога води до изтичане на метаданни и други имплементационни уязвимости; някои проучвания поставят под въпрос мащабируемостта и специфичната конфигурация за сигурност. (UCSB Computer Science+1).

3. *Mix-мрежи (Loopix, Nym и др.)*

   Силни страни: по-добра устойчивост на глобални трафик-анализи чрез batch/mix и прикриване; Loopix/Nym предлагат доказано подсилена анонимност чрез покривен трафик и *Poisson mixing*.

   Слаби страни: по-голяма латентност и по-малко са подходящи за реалновременни приложения в сравнение с Tor; сложна инфраструктура и по-високи разходи за покривен трафик (*arXiv+1*).

Как HashMouth адресира тези проблеми

1. Многопътен (*multi-path*) routing
2. Всеки голям payload се разбива на множество пакети (чънка); всеки чънк избира независим маршрут през различни възли (различни *onion* пътеки). Това намалява

вероятността един наблюдател да свърже източник и дестинация, защото различните части от съобщението минават по различни маршрути и се асемблират в крайния възел.

3. Силна *per-hop* криптография + ratchet
4. За всеки хоп: *ChaCha20-Poly1305* за AEAD, X25519 за договаряне на симетрични сесийни ключове и Ed25519 за подписване на важни мета полета (когато е необходимо). Система за „*ratcheting*“ сменя сесиите периодично, намалявайки риска при компроментиран ключ.
5. Протокол за минимален метаданен подпис (*header minimization*)
6. Всяка *onion*-обвивка съдържа само необходимата информация за следващия хоп (адрес/ID) и хеш/nonce; метаданните са минимални и криптирани.
7. Изборна, параметрируема генерация на покривен трафик (cover traffic) и padding на пакетите до фиксиран размер за затрудняване на fingerprinting / size-based анализ.
8. Комбинация с mix-подходи за чувствителни операции
9. За комуникации с високи изисквания за анонимност (например публикуване на чувствителна информация) HashMouth може да използва batch/ delay (mix) режими — търговия между латентност и анонимност — базирано на Loopix/Nym идеи. arXiv+1
10. Интегриране на *torrent* подобни *P2P* механики за throughput
11. HashMouth предлага механизъм за използване на разпределени свързвания (*swarm* подобно) в криптираната част от мрежата, при това:

    - само за предаване на вече *onion* обвити чанкове (няма директно излагане на клиентски IP в *tracker*/*DHT*);
    - *DHT*/peer-*exchange* механиките трябва да бъдат криптирани и релейни (*pendezvous*/*entry* точки), за да се избегне изтичане на IP;
    - използва се канализиране (*tunneling*) и транзитни релей пулове, за да се скрие истинската идентичност на участниците в *swarms*.

АРХИТЕКТУРА И МЕТОДОЛОГИЯ

HashMouth е изграден като peer-to-peer мрежа, в която всеки възел действа едновременно като клиент и сървър. Възлите се идентифицират чрез публични ключове, които се използват както за установяване на доверие, така и за обмен на сесийни ключове.

Основни компоненти:

- Network Layer (мрежов слой) - осигурява TCP/UDP комуникация между възлите;
- Crypto Layer (криптографски слой) - реализира onion криптиране чрез *Ed25519* и *ChaCha20-Poly1305*;
- Routing Engine - определя пътя на пакета през няколко случайни възела, преди да достигне целта;
- Node Manager - отговаря за регистрация, поддържане и прекъсване на връзки между възлите.

Всеки пакет се криптира на няколко слоя: първо с публичния ключ на крайната цел, след това с ключовете на междинните възли. При преминаване през всеки възел се разкрива само следващата дестинация, но не и източникът или крайната цел.

В основата стои Onion Routing (“луков” маршрутизиращ модел), при който:

1. Изпращачът криптира съобщението на няколко слоя (като лук);
2. Всеки възел в маршрута разшифрова само своя слой;
3. Никой възел не знае едновременно кой е източникът и кой е крайният получател.

ТЕХНИЧЕСКА РЕАЛИЗАЦИЯ

Проектът е реализиран изцяло на програмния език Go (Golang) поради неговата ефективност при работа с мрежови операции и конкурентност чрез Go рутини.

Основните файлове и модули включват:

- ***cmd/hashmouth.go*** - главен файл за стартиране на възел.
- ***network/node.go*** - обработка на връзки, маршрутизация и трансфер на пакети.
- ***crypto/crypto.go*** - реализация на многослойно (*onion*) криптиране.
- ***crypto/keys.go*** - генериране и управление на публични/частни ключове.
- ***crypto/ratchet.go*** - система за динамична смяна на ключове (*ratcheting*) за допълнителна сигурност.

План за експерименти и доказване на ефективност

- Локален тест-сет: стартиране на 10–20 възела на отделни портове/машини (VM или контейнер), сравняване на throughput и latency при: (a) директен Tor-like single-path; (b) HashMouth multi-path; (c) HashMouth + swarm-подобна подсистема.
- Метрики: end-to-end latency, percent packets delivered, throughput (MB/s), entropy на наблюдаван трафик (колкото по-висок — толкова по-трудно за fingerprinting), и resilience срещу node compromise (симулирано).
- Анализ на трафикови атаки: прилагане на отворени техники за трафик анализ (website fingerprinting, flow correlation) за измерване на информацията, която остава видима при наблюдател. Сравняване със съществуващи резултати (Tor / Loopix).

ЗАКЛЮЧЕНИЕ И БЪДЕЩА РАБОТА

HashMouth показва как чрез сравнително прост код може да се реализира сложна концепция за анонимна и криптографски защитена комуникация. Демонстрира как е възможно да се обединят принципите на TOR и P2P технологиите в нов хибриден модел, който предлага:

- по-висока скорост**,**
- по-голяма анонимност
- по-добра съвместимост с реалния интернет.

Бъдеща работа:

- Добавяне на DHT (Distributed Hash Table) за откриване на възли.
- Поддръжка на UDP транспорт за по-висока производителност.
- Имплементация на GUI интерфейс за визуализация на мрежата.
- Възможност за изпращане на файлове и групови съобщения.

ИЗТОЧНИЦИ

1. Golang Official Documentation — https://go.dev/doc/
2. Golang Crypto Libraries — https://pkg.go.dev/golang.org/x/crypto
3. Bernstein, D. J. *Curve25519: New Diffie-Hellman speed records*, 2006.
4. I2P Network Overview — https://geti2p.net
5. Tor Project — https://www.torproject.org
6. RFC 8439 — *ChaCha20 and Poly1305 for IETF protocols*
7. NaCl Cryptography Library — https://nacl.cr.yp.to